MoleHole: Real-time Black Hole Simulation

Gabriel Martin and Leonhard Neuhold

October 11, 2026

Abstract

bla bla bla

Contents

1 Introduction **1**

2 Math - Theory **1**
2.1 Integration Methods . . . 1
2.1.1 Monte Carlo Integration . . . 1

3 Rendering - Theory **1**
3.1 Raytracing . . . 1
3.1.1 Overview . . . 1
3.1.2 Acceleration Structures . . . 1
3.2 Radiometry and Surface Scattering . . . 1
3.2.1 Radiometry and Photometry . . . 2
3.2.2 Bidirectional Reflectance Distribution Function (BRDF) . . . 2
3.2.3 Bidirectional Transmittance Distribution Function (BTDF) . . . 2
3.2.4 Bidirectional Scattering Distribution Function (BSDF) . . . 2
3.2.5 Isotropic vs Anisotropic Scattering . . . 3
3.3 The Rendering Equation . . . 3
3.3.1 The Equation . . . 3
3.3.2 Spherical Integration . . . 4
3.3.3 Color Spectrum . . . 4
3.3.4 Energy Conservation (White Furnace Test) . . . 4

4 Physics - Theory **4**
4.1 Numbers and Constants . . . 4
4.1.1 The Modern SI . . . 4
4.1.2 Integer Overflows . . . 5
4.1.3 Floating-Point Representation and Numerical Errors . . . 5
4.1.4 Simulation Engine Constants Table . . . 5

Acronyms **5**

1 Introduction

TODO

2 Math - Theory

2.1 Integration Methods

2.1.1 Monte Carlo Integration

3 Rendering - Theory

3.1 Raytracing

3.1.1 Overview

TODO: Ray and camera math, ray generation, ray intersection, spheres, planes, triangles, etc.

3.1.2 Acceleration Structures

Acceleration structures are essential for reducing the computational complexity of ray-tracing from $O(N \cdot M)$ to a logarithmic scale. They utilize spatial or object subdivision to quickly cull geometry that a ray cannot intersect. This section is based on lecture 1 of a course from the TU Wien Faculty of Informatics, Institute of Visual Computing & Human-Centered Technology, Research Unit of Computer Graphics[9].

Bounding Volumes Bounding volumes simplify complex geometry into basic shapes that are computationally cheap to test against rays. Common types include:

- **Axis-Aligned Bounding Box (AABB)**: Defined by minimum and maximum extents along the coordinate axes (x, y, z). They are fast to build and test (using the slab method) but may fit underlying geometry poorly if the object is rotated, leading to more false positives during traversal.
- **Oriented Bounding Box (OBB)**: Similar to AABBs but aligned with the object's principal axes rather than the world axes. They provide a much tighter fit, reducing the number of false positive intersections, but are more expensive to build and test.
- **Bounding Sphere**: Defined by a center point and a radius. They are rotationally invariant and offer the fastest intersection test (procedural spheres work well with way math as described in Section 3.1.1), but often provide a bad fit for complex objects because the spheres will inevitably overlap.

Spatial vs. Object Subdivision

- **Spatial Subdivision**: The space itself is divided into cells like Grids, Octrees, or K-Dimensional Trees (K-d Trees). Objects that span boundaries must be referenced in multiple nodes, potentially increasing memory usage.
- **Object Subdivision**: The set of objects is partitioned into a hierarchy, most notably the Bounding Volume Hierarchy (BVH). A BVH is a tree structure where each node represents a bounding volume that encompasses all geometry in its children. Unlike spatial subdivision, each geometric primitive belongs to exactly one leaf node. Bounding volumes of sibling nodes can overlap, but memory consumption is generally lower and predictable ($2N-1$ nodes for N primitives).

Building Strategies Top-down construction is a common approach, particularly for BVHs. The set of primitives is recursively split into child nodes until a leaf criterion (e.g., max triangles per node) is met. The quality of the tree depends heavily on the split strategy:

- **Spatial Median**: Splits the node at the spatial center of its bounding box. Simple but can result in unbalanced trees if geometry is clustered.
- **Object Median**: Splits the list of objects into two equal halves based on their centroids. Ensures balanced trees but may result in large overlapping bounding boxes.
- **Surface Area Heuristic (SAH)**: A greedy algorithm that attempts to minimize the expected traversal cost. It evaluates potential splits based on the probability of a ray hitting a child node (proportional to its surface area) and the cost of traversing it.

  $$Cost(b) = SA_L(b) \cdot N_L(b) + SA_R(b) \cdot N_R(b)$$

  where SA is the surface area and N is the number of primitives.

Comparison The choice of acceleration structure involves trade-offs between build time, memory usage, and traversal speed, as shown in Table 1.

3.2 Radiometry and Surface Scattering

This section covers the fundamental concepts of radiometry, photometry, and surface scattering functions that are essential for understanding light transport in rendering.

Table 1: Comparison of Spatial Acceleration Structures[9]

Structure	Memory	Build Time	Traversal Quality
Regular Grid	Low – High	Low	Poor (non-uniform)
Quad/Octree	Low – High	Low	Good
K-d Tree	Low – High	Low – High	Good – Excellent
BVH	Low	Low – High	Good – Excellent

3.2.1 Radiometry and Photometry

It is important to understand the difference between radiometry (physical measurement of Electro-Magnetic (EM) radiation) and photometry (light as it is perceived by the human eye). Both systems define their own set of units. The conversion between the two is defined by the spectral luminous efficiency function $V(\lambda)$, which peaks at approximately 555 nm (green). This is why colors like green and yellow appear brighter to the human eye compared to colors like blue.

Table 2 shows the key radiometric quantities, and Table 3 lists their photometric equivalents. Note that while rendering calculations are performed using radiometric units (specifically Radiance), output devices (displays) and human perception operate in the photometric domain (Luminance). [9].

3.2.2 Bidirectional Reflectance Distribution Function (BRDF)

The Bidirectional Reflectance Distribution Function (BRDF) defines how a material scatters incoming light from direction ω_i into outgoing direction ω_o at a given surface point x [12]. Formally the BRDF is the differential ratio of reflected radiance $\mathrm{d}L_r$ to incident irradiance $\mathrm{d}E_i$:

$$f_r(x, \omega_i, \omega_o) = \frac{\mathrm{d}L_r(x, \omega_o)}{\mathrm{d}E_i(\omega_i)} \tag{1}$$

$$= \frac{\mathrm{d}L_r(x, \omega_o)}{L_o(y_i, -\omega_i)\,|\cos\theta_i|\,\mathrm{d}\omega_i} \tag{2}$$

The unit of the BRDF is radiance divided by irradiance, which is sr^{-1} (inverse steradians).

In terms of fundamental flux quantities, the BRDF can also be expressed as:

$$f_r(x, \omega_i, \omega_o) = \frac{\mathrm{d}^3\Phi_r}{\mathrm{d}^2\Phi_i\,\mathrm{d}\omega_o\,|\cos\theta_o|} \tag{3}$$

The BRDF must also satisfy the following requirements:

- **Positivity**: Light cannot be negative, so it must satisfy $f_r(x, \omega_i, \omega_o) \geq 0$ for all ω_i, ω_o
- **Helmholtz Reciprocity**: States that a ray and its reverse ray must follow the same optical path (e.g. reflections, refractions) $f_r(x, \omega_i, \omega_o) = f_r(x, \omega_o, \omega_i)$
- **Energy Conservation**: The cosine weighted integral of the BRDF over the hemisphere must be 1 or less for all ω_o:

$$\int_\Omega f_r(x, \omega_i, \omega_o)|\cos\theta_o|\mathrm{d}\omega_o \leq 1$$

3.2.3 Bidirectional Transmittance Distribution Function (BTDF)

While the BRDF covers reflection, the Bidirectional Transmittance Distribution Function (BTDF) handles light that passes through a surface (transmission). The lower hemisphere Ω^- is used for incoming light (BRDF), and the upper hemisphere Ω^+ is used for outgoing light (BTDF). Unlike reflection, the BTDF is not symmetric, as light can be transmitted in one direction but not the other, and when light transmits across an interface between two media, with different refractive indices n_1 and n_2, the BTDF is scaled by the ratio of the squared indices of refraction $(n_2/n_1)^2$ according to Snell's law. [2, 12]

$$f_t(\omega_i, \omega_t) = \frac{dL_t(\omega_t)}{dE_i(\omega_i)} = \frac{dL_t(\omega_t)}{L_i(\omega_i)|\cos\theta_i|d\omega_i}$$

3.2.4 Bidirectional Scattering Distribution Function (BSDF)

The Bidirectional Scattering Distribution Function (BSDF) unifies reflection and transmission into a single function over the entire sphere of directions 4π steradians, logically representing the sum of f_r and f_t. [1, 13]

$$f_s(\omega_i, \omega_o) = f_r(\omega_i, \omega_o) + f_t(\omega_i, \omega_t)$$

Depending on whether the incident and outgoing directions are on the same side of the surface (reflection) or opposite sides (transmission), the BSDF decomposes into two distinct components, because light cannot simultaneously reflect and transmit along the exact same outgoing vector ω_o at a sharp boundary point. Therefore, the general BSDF is defined piecewise as:

Table 2: Radiometric Quantities (Physical) [9]

Quantity	Symbol	Unit	Description
Radiant Energy	Q_e	Joule [J]	Total energy of radiation
Radiant Flux (Power)	Φ_e	Watt [W]	Energy per unit time (J/s)
Radiant Intensity	I_e	Watt/sr [W/sr]	Flux per unit solid angle
Irradiance	E_e	Watt/m^2 [W/m^2]	Incident flux per unit area
Radiance	L_e	Watt/(sr $\cdot$ m^2)	Flux per unit projected area per unit solid angle

Table 3: Photometric Quantities (Perceptual) [9]

Quantity	Symbol	Unit	Description
Luminous Energy	Q_v	Talbot [T]	Perceived energy of light
Luminous Flux	Φ_v	Lumen [lm]	Perceived power
Luminous Intensity	I_v	Candela [cd]	Flux per unit solid angle (lm/sr)
Illuminance	E_v	Lux [lx]	Incident flux per unit area (lm/m^2)
Luminance	L_v	Nit [cd/m^2]	Perceived brightness of a surface

$$f_s(x, \omega_i, \omega_o) = \begin{cases} f_r(x, \omega_i, \omega_o), & \text{reflection} \\ f_t(x, \omega_i, \omega_o), & \text{transmission} \end{cases} \quad (4)$$

3.2.5 Isotropic vs Anisotropic Scattering

Scattering at a point depends on 4 angular dimensions: the incident direction (θ_i, ϕ_i) and the outgoing direction (θ_o, ϕ_o). [13, 14]

Isotropic Scattering: A surface is isotropic if its scattering properties are invariant under rotation about the surface normal n. Spinning the object around its normal does not change the appearance of the surface. Therefore the scattering function no longer depends on the azimuthal angles ϕ_i and ϕ_o, and the BSDF reduces to a function of only the polar angles θ_i and θ_o: $f_s(\theta_i, \theta_o, \Delta\phi)$ Examples of isotropic materials include Lambertian surfaces, metals, dielectrics, glass, and some plastics.

Anisotropic Scattering: Anisotropic surfaces have scattering properties that vary with rotation about the surface normal. This leaves the BSDF as a 4D function of both the polar and azimuthal angles: $f_s(\theta_i, \phi_i, \theta_o, \phi_o)$. Anisotropy commonly results from an oriented microscopic structure. For example mie scattering from aligned fibers, water droplets, brushed metals, or dust.

In practice, anisotropic models often use a tangent direction t in addition to the surface normal n to describe this orientation. The tangent and bitangent define a local frame in which the microfacets or scattering lobes can have different roughness values along different axes. When the roughness is equal in both tangent directions, the model reduces to the isotropic case.

3.3 The Rendering Equation

The fundamental mathematical model for simulating light in physically-based rendering is the Rendering Equation, introduced by Kajiya in 1986[8]. It describes the equilibrium distribution of radiance in a scene.

3.3.1 The Equation

The Rendering Equation expresses the total spectral radiance L_o leaving a point x in direction ω_o as the sum of emitted radiance L_e and reflected radiance:

$$L_o(x, \omega_o) = L_e(x, \omega_o) + \int_\Omega f_r(x, \omega_i, \omega_o) L_i(x, \omega_i)(\omega_i \cdot n) d\omega_i \quad (5)$$

where:

- $L_o(x, \omega_o)$ is the total outgoing radiance.
- $L_e(x, \omega_o)$ is the emitted radiance (for light sources).
- Ω is the unit hemisphere centered at normal n containing all incoming directions ω_i.
- $f_r(x, \omega_i, \omega_o)$ is the BRDF, defining how light is scattered.
- $L_i(x, \omega_i)$ is the incoming radiance from direction ω_i.
- $(\omega_i \cdot n)$ is the geometry term (Lambert's cosine law), accounting for the spreading of light over the surface area (often written as $\cos\theta_i$).

Usage of Radiance (L) Radiance (L) is the central quantity in rendering for several reasons:

1. **Invariance**: Radiance is constant along a ray in a vacuum. This implies $L_i(x, \omega) = L_o(y, -\omega)$ where y is the visible surface point. This property enables ray tracing: to find the radiance reaching the camera, we simply trace a ray into the scene and calculate the radiance leaving the intersected surface.

2. **Sensor Response**: Camera sensors and the human eye respond to radiance (integrated over the pixel area and solid angle).

3. **Light Transport**: It describes the flow of light energy through space.

The recursive nature of L_i (which is the L_o from another point) makes this integral equation difficult to solve analytically. Therefore, most implementations use numerical methods like Monte Carlo integration (see 2.1.1).

3.3.2 Spherical Integration

To solve the integral numerically, we often parameterize the hemisphere Ω using spherical coordinates (θ, ϕ). The differential solid angle $d\omega_i$ changes variables as follows:

$$d\omega = \sin\theta d\theta d\phi \tag{6}$$

Substituting this into the rendering equation gives the explicit double integral form:

$$\int_\Omega (\dots) d\omega_i = \int_0^{2\pi} \int_0^{\frac{\pi}{2}} L_i(x, \theta, \phi) \cdot f_r(x, \theta, \phi, \omega_o) \cos\theta \sin\theta d\theta d\phi \tag{7}$$

Here, θ is the polar angle (from the normal n, range $[0, \pi/2]$) and ϕ is the azimuth angle (range $[0, 2\pi]$). The term $\cos\theta$ comes from the geometry term $(\omega_i \cdot n)$, and $\sin\theta$ comes from the differential solid angle measure.

3.3.3 Color Spectrum

Radiance L varies with wavelength ($L(\lambda)$). In computer graphics, strictly accurate rendering would require integrating over the entire visible spectrum. However, for efficiency, we typically sample this spectrum at three discrete wavelengths corresponding to Red, Green, and Blue (RGB). This mimics the tristimulus response of the human eye. While sufficient for most purposes, this approximation can fail for phenomena like dispersion, fluorescence or metamerism resulting in inaccurate results.

3.3.4 Energy Conservation (White Furnace Test)

A physically-based material must conserve energy: it cannot reflect more light than it receives. The "White Furnace Test" is a thought experiment (or unit-test) used to verify this. If a non-absorbing object is placed in an environment with uniform radiance $L_i = 1$ everywhere, the outgoing radiance L_o must also be exactly 1 (as long as no other form of light is emitted). For a perfectly diffuse (Lambertian) surface with albedo ρ, the BRDF is constant $f_r = c$. Integrating the cosine term over the hemisphere yields π:

$$\int_\Omega \cos\theta d\omega = \pi$$

Therefore, to ensure $L_o = \rho L_i$, we must normalize the BRDF:

$$f_{r,\text{diffuse}} = \frac{\rho}{\pi}$$

The factor of $1/\pi$ is essential for energy conservation.

4 Physics - Theory

4.1 Numbers and Constants

High-fidelity physical simulation engines require precise numerical representations of physical constants and quantities. These representations must accurately reflect the true values of the constants as established by international standards.

4.1.1 The Modern SI

Following the 2019 revision of the International System of Units (SI) [3, 11], the definitions moved from artifact-based definitions to exact numerical definitions based on fundamental constants of nature.

Constants can be categorized into two types:

1. **Defined Constants:** These are constants whose numerical values are fixed by definition in the SI standard. They have zero measurement uncertainty. These include c, h, e, k_B, N_A, and others. These values are hardcoded as exact literals to the maximum precision supported by the data type. Many other constants, such as μ_0 and ϵ_0, are derived from these fixed constants and are also treated as exact values.

2. **Experimental Constants:** These constants are determined through empirical measurement and carry experimental uncertainties[10]. These include the gravitational constant (G), the fine-structure constant (α), and particle/object masses (e.g., electron mass m_e, hydrogen mass m_H, solar mass $M_\odot$). While

CODATA provides recommended values along with their uncertainties, they are treated as floating-point values subject to experimental updates.

4.1.2 Integer Overflows

Discrete quantities are represented as binary signed or unsigned integers, which can overflow when their values exceed the maximum or minimum representable values. The range of an n-bit signed integer is $[-2^{n-1}, 2^{n-1} - 1]$, while an unsigned integer has a range of $[0, 2^n - 1]$.

When an integer exceeds its maximum value, it wraps around to the minimum value (and vice versa). Therefore, it is crucial to use sufficiently large integer types and/or implement overflow checking mechanisms. This requires careful consideration of expected number ranges.

4.1.3 Floating-Point Representation and Numerical Errors

Computers approximate real numbers using finite-precision representations governed by the IEEE 754 standard for floating-point arithmetic[7]. Compared to integer representations, floating-point numbers can represent a much wider range of values, by using a normalized scientific notation with a fixed number of significant digits (the mantissa) and an exponent.

Floating-point systems introduce fundamental error sources[6]:

- **Rounding and Machine Epsilon:** Real numbers cannot be represented with infinite precision. The spacing between adjacent representable floating-point numbers is bounded by the Machine Epsilon ϵ_{mach}. Operations on physical quantities introduce rounding errors that accumulate over multi-step time integrations (e.g., Verlet or Runge-Kutta integrators). While single precision (32-bit) is the de facto standard for most programing languages, it is insufficient for large-scale simulations with both high number ranges and small deltas. Double precision (64-bit) and quadruple precision (128-bit) are also available on modern processors, but come with increased memory usage and computational cost. See Table 4 for a comparison of IEEE 754 formats.

- **Catastrophic Cancellation:** Subtracting two nearly equal floating-point numbers results in loss of significant digits. This frequently occurs when computing relative position vectors between close particles ($\vec{r}_1 - \vec{r}_2$), leading to increased relative errors.

- **Underflow, Overflow, and NaN:** Exceeding the dynamic range of the exponent field results in overflow (yielding $\pm\infty$) or underflow (yielding subnormal numbers or zero), which can corrupt calculations. A floating-point values might also hold the special value NaN (Not a Number) to indicate undefined or unrepresentable results, such as $0/0$ or $\infty - \infty$.

4.1.4 Simulation Engine Constants Table

To ensure reproducibility, determinism, and maximum numerical precision, the simulation engine initializes its internal physical constants using exact 2019 SI definitions[3, 11] and standard CODATA evaluated values[10] stored as 64-bit double-precision (or higher) floating-point numbers. The excact values can be referenced in the source code under `TODO`.

Acronyms

AABB Axis-Aligned Bounding Box. 1

BRDF Bidirectional Reflectance Distribution Function. 2, 3

BVH Bounding Volume Hierarchy. 1, 2

EM Electro-Magnetic. 2

K-d Tree K-Dimensional Tree. 1, 2

OBB Oriented Bounding Box. 1

SAH Surface Area Heuristic. 1

References

[1] Frederick O. Bartell, Eustace L. Dereniak, and William L. Wolfe. "The theory and measurement of bidirectional scattering distribution function (BSDF)". In: *Radiation Scattering in Optical Systems*. Vol. 0257. SPIE. 1980, pp. 154–160.

[2] Max Born et al. *Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction of Light*. 7th ed. Cambridge University Press, 1999.

[3] Bureau International des Poids et Mesures. *The International System of Units (SI)*. 9th. BIPM, 2019. ISBN: 978-92-822-2272-0. DOI: `https://doi.org/10.59161/AUEZ1291`.

Table 4: Comparison of IEEE 754 floating-point formats[7]

Format	**Digits**	**(ϵ_{mach})**	**Dynamic Range**	**Relative Error (50th / 99th)**
`f32`	$6-7$	$\approx 1.19 \times 10^{-7}$	1.18×10^{-38} to 3.40×10^{38}	$\sim 10^{-7}$ / $\sim 10^{-5}$
`f64`	$15-17$	$\approx 2.22 \times 10^{-16}$	2.23×10^{-308} to 1.79×10^{308}	$\sim 10^{-16}$ / $\sim 10^{-14}$
`f128`	34	$\approx 1.92 \times 10^{-34}$	3.36×10^{-4932} to 1.18×10^{4932}	$\sim 10^{-34}$ / $\sim 10^{-32}$

[4] Paul Adrien Maurice Dirac. *The Principles of Quantum Mechanics*. International series of monographs on physics. Clarendon Press, 1981. ISBN: 9780198520115.

[5] Albert Einstein. "Zur Elektrodynamik bewegter Körper. (German) [On the electrodynamics of moving bodies]". In: *Annalen der Physik* 322.10 (1905), pp. 891–921. DOI: http://dx.doi.org/10.1002/andp.19053221004.

[6] David Goldberg. "What every computer scientist should know about floating-point arithmetic". In: *ACM Comput. Surv.* 23.1 (Mar. 1991), pp. 5–48. ISSN: 0360-0300. DOI: 10.1145/103162.103163. URL: https://doi.org/10.1145/103162.103163.

[7] "IEEE Standard for Floating-Point Arithmetic". In: *IEEE Std 754-2019 (Revision of IEEE 754-2008)* (2019), pp. 1–84. DOI: 10.1109/IEEESTD.2019.8766229.

[8] James T. Kajiya. "The rendering equation". In: *SIGGRAPH Comput. Graph.* 20.4 (Aug. 1986), pp. 143–150. ISSN: 0097-8930. DOI: 10.1145/15886.15902. URL: https://doi.org/10.1145/15886.15902.

[9] Bernhard Kerbl and Adam Celarek. *Rendering (VU) - Summer Semester 2020*. TU Wien, Institute of Visual Computing & Human-Centered Technology. Accessed: 2024-05-22. 2020. URL: https://www.cg.tuwien.ac.at/courses/Rendering/VU.SS2020.html.

[10] Peter J. Mohr, David B. Newell, and Barry N. Taylor. "CODATA recommended values of the fundamental physical constants: 2014". In: *Rev. Mod. Phys.* 88 (3 Sept. 2016), p. 035009. DOI: 10.1103/RevModPhys.88.035009. URL: https://link.aps.org/doi/10.1103/RevModPhys.88.035009.

[11] David Newell and Eite Tiesinga. *The International System of Units (SI), 2019 Edition*. en. Aug. 2019. DOI: https://doi.org/10.6028/NIST.SP.330-2019.

[12] F. E. Nicodemus et al. "Geometrical considerations and nomenclature for reflectance". In: *Radiometry*. USA: Jones and Bartlett Publishers, Inc., 1992, pp. 94–145. ISBN: 0867202947.

[13] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. 4th. Morgan Kaufmann, 2023. ISBN: 978-0128006450. URL: https://www.pbr-book.org/4ed.

[14] Gregory J. Ward. "Measuring and modeling anisotropic reflection". In: *Proceedings of the 19th Annual Conference on Computer Graphics and Interactive Techniques*. SIGGRAPH '92. New York, NY, USA: Association for Computing Machinery, 1992, pp. 265–272. ISBN: 0897914791. DOI: 10.1145/133994.134078. URL: https://doi.org/10.1145/133994.134078.